

Helm Study Notes for CKA / CKAD / KCNA

PART 1: FOUNDATIONS

Chapter 1 — What is Helm and Why Does It Exist?

Helm is a **Kubernetes package manager** that makes deploying, upgrading, and managing applications on Kubernetes easier, repeatable, and safer. It is the tool that sits between raw YAML manifests and cluster operations, bringing structure, templating, versioning, and lifecycle management to Kubernetes workloads.

The problem before Helm: managing raw Kubernetes YAML at scale

Before Helm, the default way to deploy applications was to write raw Kubernetes manifests and apply them with `kubectl apply -f`. That works well for one-off resources, but it becomes painful when:

- you need to deploy the same application to multiple environments
- you want reusable configuration with environment-specific values
- you need atomic upgrades and rollbacks
- you want a packaged, versioned application that other teams can install

Raw YAML is declarative, but it is not a package. It does not have a standard structure for application metadata, dependencies, or release history.

NOTE: For CKA/CKAD/KCNA, remember that Helm is not just a templating engine. It also manages release state and lifecycle operations.

What a package manager means in the Kubernetes context

A package manager makes it easier to consume, install, and upgrade software by packaging it with metadata and standardized commands.

Compare:

- apt / yum for Linux packages
- npm for Node.js packages
- pip for Python packages
- Helm for Kubernetes applications

In Kubernetes, a package manager must understand:

- configuration values per deployment
- templated YAML resources
- release lifecycle (install, upgrade, rollback)
- dependency resolution

Helm brings this missing layer to Kubernetes, allowing developers to treat a set of manifests as a reusable application package.

Helm as the "missing layer" between kubectl and production deployments

kubectl is the low-level tool to talk to the Kubernetes API. It applies manifests, inspects objects, and manages resources. Helm adds a higher-level layer:

- package definition via charts
- parameterized inputs via values
- release versioning and history
- install/upgrade/rollback workflows
- artifact distribution through charts and repos

This is why Helm is often compared to an "apt for Kubernetes": it manages application bundles, not individual objects.

The three core concepts: Charts, Releases, Repositories

- **Chart:** A package of Kubernetes manifests and metadata, usually stored as a directory or .tgz archive. A chart defines a reusable application.

- **Release:** A specific installation of a chart into a cluster. Each release has a name and a revision history.
- **Repository:** A catalog of charts served over HTTP, similar to an apt repository or npm registry.

These concepts are the foundation of Helm usage.

Helm v2 vs Helm v3 – what changed, why Tiller was removed, security implications

Aspect	Helm v2	Helm v3	Why it changed
Server-side component	Tiller	None	Tiller introduced complexity and security risk
Release storage	ConfigMap or Secret in kube-system	Secret/ConfigMap in release namespace by default	Release state is now namespace-scoped and removes cluster-wide privilege needs
RBAC	Tiller needed cluster role binding	Helm client uses user's kubeconfig credentials	Better security, no centralized service account
Security	Elevated privileges, poor auditability	Uses normal Kubernetes RBAC	More secure for CKA/CKAD/KCNA exam contexts
Complexity	More components to install and manage	Simpler client-only architecture	Easier for CI/CD and GitOps

EXAM TIP: For modern Kubernetes certification, always assume Helm v3 unless a question explicitly states Helm v2.

Tiller was removed because it acted as an intermediary with broad cluster privileges and stored secrets on behalf of users. This created security vulnerabilities including:

- excessive permissions beyond the user's own rights
- hidden cluster-wide state and access control issues
- audit gaps because operations were performed by Tiller rather than the user

By removing Tiller, Helm v3 uses the same kubeconfig-based credentials as kubectl, which aligns

with Kubernetes RBAC and reduces attack surface.

When NOT to use Helm (and what to use instead)

Helm is powerful, but it is not always the right choice.

Use something else when:

- you need only simple patching of existing manifests → use **Kustomize**
- you have a small cluster with one-off resources → use **raw kubectl YAML**
- you need operator-driven lifecycle management for complex stateful applications → use the **Operator pattern**

Alternatives:

- **Kustomize**: great for composition and customization of existing resources without templating.
- **Raw kubectl**: good for learning, simple deployments, or single-manifest cases.
- **Operators**: good for applications that require continuous reconciliation and custom control loops.

COMMON MISTAKE: Choosing Helm for every workload. If you only need to apply a few static manifests or you are managing CRDs with complex semantics, Helm may be overkill.

Chapter 1 summary

- Helm is a Kubernetes package manager that packages manifests into reusable charts.
- It solves templating, values management, release lifecycle, and dependency resolution.
- Charts, releases, and repositories are the core Helm concepts.
- Helm v3 removed Tiller to improve security and simplify architecture.
- Use raw kubectl, Kustomize, or Operators when Helm is not the best fit.

Chapter 2 — Architecture Deep Dive

Helm client architecture (no server-side component in v3)

Helm v3 is a pure client-side tool. When you run `helm install`, the Helm client:

1. reads the chart and values
2. renders templates locally
3. contacts the Kubernetes API server using kubeconfig credentials
4. applies the rendered resources
5. stores release metadata in the cluster

This means Helm v3 does not require a long-running server-side component. It is simpler and more secure than Helm v2.

How Helm communicates with the Kubernetes API server

Helm uses the same client-go library as `kubectl`. The request flow is:

1. Helm reads `$KUBECONFIG` or defaults to `~/.kube/config`
2. It determines the current context and cluster endpoint
3. Helm sends standard Kubernetes API requests to the API server
4. It creates or updates resources, and reads/writes release storage objects

Helm does not use a special protocol. It is simply a Kubernetes client with release metadata handling.

Release storage mechanism: Kubernetes Secrets, ConfigMaps, memory, SQL

Helm uses a **storage driver** to save release state. Common drivers include:

- secret (default)
- configmap
- memory (for testing)
- sql (using a SQL database, rarely used)

Secrets:

- default in Helm v3

- stores release metadata as base64-encoded YAML
- recommended for most use cases
- note: base64 is not encryption

ConfigMaps:

- alternative storage driver
- useful when Secrets are restricted or unavailable
- less secure because data is not base64-encoded as secret data fields

Memory:

- temporary storage for test environments
- data does not persist across Helm client runs

SQL:

- advanced use case for external storage
- not common for exam preparation, but supported by the Helm driver architecture

EXAM TIP: When asked where Helm stores release metadata, answer: by default in Kubernetes Secrets in the release namespace.

The role of \$KUBECONFIG and how Helm inherits cluster context

Helm uses \$KUBECONFIG just like kubectl. If the environment variable is set, Helm reads it. Otherwise it falls back to ~/.kube/config or the in-cluster service account if running inside Kubernetes.

You can override the current cluster/context by using:

```
export KUBECONFIG=~/.kube/my-cluster.yaml  
helm install myapp ./chart
```

Helm also supports --kubeconfig and --kube-context flags for explicit cluster selection.

Helm's three storage paths (Cache, Config, Data)

Helm stores client-side state in three directories:

- HELM_CACHE_HOME: cached downloaded charts and repository data
- HELM_CONFIG_HOME: configuration files such as repositories, plugins, and repositories.yaml
- HELM_DATA_HOME: local data such as plugins and local registry cache

OS	HELM_CACHE_HOME	HELM_CONFIG_HOME	HELM_DATA_HOME
Linux	~/.cache/helm	~/.config/helm	~/.local/share/helm
macOS	~/Library/Caches/helm	~/Library/Preferences/helm	~/Library/Application Support/helm
Windows	%LOCALAPPDATA%\helm\cache	%APPDATA%\helm	%APPDATA%\helm\data

These directories are controlled by environment variables and can be customized.

Environment variables reference

Here are the common \$HELM_* environment variables and when to use them:

- HELM_CACHE_HOME
 - custom location for cached charts and repository index files
 - useful if you want shared cache across users or a custom disk location
- HELM_CONFIG_HOME
 - custom location for Helm configuration files
 - contains repositories.yaml, repository/cache, and plugin metadata
- HELM_DATA_HOME
 - custom location for Helm data and plugin storage
 - contains local registry cache and plugin data
- HELM_DRIVER
 - selects storage driver: secret, configmap, memory, sql
 - example: export HELM_DRIVER=configmap
- HELM_KUBECONTEXT
 - sets the kubeconfig context used by Helm
 - overrides --kube-context

- **HELM_NAMESPACE**
 - default namespace for Helm operations when `--namespace` is not provided
 - useful in CI to avoid repeating namespace flags
- **HELM_DEBUG**
 - enables debug logging in Helm
 - set to 1 or true for noisy debugging output
- **HELM_NO_PLUGINS**
 - disable plugin loading when set to 1
 - useful if a plugin is causing issues
- **HELM_PLUGINS**
 - override the plugin directory
 - useful for isolated plugin tests
- **HELM_REPOSITORY_CACHE**
 - exact path to the repository cache directory
- **HELM_REPOSITORY_CONFIG**
 - exact path to the repository configuration file

COMMON MISTAKE: Using `HELM_DRIVER=memory` in production. It does not persist release state and should only be used for local testing.

Chapter 2 summary

- Helm v3 is client-only and communicates with Kubernetes using kubeconfig.
- Release state is stored using a driver, defaulting to Kubernetes Secrets.
- `$KUBECONFIG` controls cluster context just as with `kubectl`.
- Helm stores client-side files in cache, config, and data directories.
- Helm environment variables let you override configuration paths and runtime behavior.

Chapter 3 — Installation & Environment Setup

Installing Helm

Helm can be installed in several ways. Each method has tradeoffs.

Snap:

`sudo snap install helm --classic`

- Pros: easy on Ubuntu-like systems
- Cons: snap packaging may lag behind latest releases, not available everywhere

apt:

`curl https://baltocdn.com/helm/signing.asc | sudo apt-key add -`

`sudo apt-get install apt-transport-https --yes`

`echo "deb https://baltocdn.com/helm/stable/debian/ all main" | sudo tee /etc/apt/sources.list.d/helm-stable-debian.list`

`sudo apt-get update`

`sudo apt-get install helm`

- Pros: integrates with system package manager
- Cons: package versions may lag, requires repository setup

Homebrew (macOS/Linux):

`brew install helm`

- Pros: widely used, easy to upgrade
- Cons: depends on Homebrew being installed

Binary download:

`curl -LO https://get.helm.sh/helm-v3.11.0-linux-amd64.tar.gz`

`tar -zxvf helm-v3.11.0-linux-amd64.tar.gz`

`sudo mv linux-amd64/helm /usr/local/bin/helm`

- Pros: precise control over version
- Cons: manual update process

Install script:

`curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash`

- Pros: fast and simple
- Cons: less control over installation location, trust script source

Verifying installation

Check Helm is installed and note the client/server information:

helm version

Inspect Helm environment and directories:

helm env

EXAM TIP: helm version is one of the first commands to run after installation. On an exam, use it to verify the client and identify if Helm is v2 or v3.

Configuring KUBECONFIG for multi-cluster environments

When you manage multiple clusters, KUBECONFIG can contain multiple contexts.

List contexts:

kubectl config get-contexts

Switch context:

kubectl config use-context my-cluster

Explicitly set KUBECONFIG for Helm:

```
export KUBECONFIG=~/.kube/config:~/.kube/other-config
```

```
helm install myapp ./chart --kube-context=my-cluster
```

COMMON MISTAKE: Forgetting to set KUBECONFIG or --kube-context before installing. On an exam, always verify you're on the right cluster first.

Shell autocompletion

Helm supports shell completion for Bash, Zsh, and Fish.

Bash:

`source <(helm completion bash)`

Zsh:

`source <(helm completion zsh)`

Fish:

`helm completion fish | source`

Persist completion by adding the appropriate command to your shell profile.

Helm plugins: what they are and how to install

Helm plugins extend Helm commands.

List installed plugins:

`helm plugin list`

Install a plugin:

`helm plugin install https://github.com/databus23/helm-diff`

Plugins are stored under `HELM_DATA_HOME/plugins`.

EXAM TIP: Plugins are useful in real life, but certification exams only test core Helm commands unless they explicitly mention plugins.

Chapter 3 summary

- Install Helm with snap, apt, brew, binary, or script.
- Verify with `helm version` and `helm env`.
- Use `KUBECONFIG` and `--kube-context` for multi-cluster work.
- Enable shell completion for faster command entry.
- Helm plugins extend functionality, but core exam focus remains on built-in Helm.

PART 2: CORE CONCEPTS IN DEPTH

Chapter 4 — Charts: Anatomy and Structure

What a Chart is

A **Chart** is a packaged, templated Kubernetes application. It contains a set of Kubernetes manifests, metadata, default values, and optionally dependencies.

Charts are reusable: the same chart can be installed multiple times with different values, producing different releases.

Full directory structure of a Helm chart

A typical chart looks like this:

```
nginx-deploy-chart/  
├── Chart.yaml  
├── values.yaml  
├── charts/  
├── templates/  
│   ├── deployment.yaml  
│   ├── service.yaml  
│   ├── ingress.yaml  
│   ├── _helpers.tpl  
│   ├── NOTES.txt  
│   └── tests/  
│       └── test-connection.yaml  
└── .helmignore
```

Chart.yaml

Chart.yaml is the chart manifest. Every field documents metadata that Helm uses.

Example:

```
apiVersion: v2  
name: nginx-deploy-chart  
description: A Helm chart for deploying nginx
```

type: application
version: 0.1.0
appVersion: 1.23.1
keywords:
- nginx
- web
home: https://example.com/nginx
sources:
- https://github.com/yourorg/nginx-deploy-chart
maintainers:
- name: alice
email: alice@example.com
dependencies:
- name: redis
version: 14.0.0
repository: https://charts.bitnami.com/bitnami
annotations:
category: webserver
Field details:

- apiVersion: Helm chart API version (v1 or v2). Use v2 for Helm v3 charts.
- name: chart name, used in package filenames and release identification.
- version: chart version, follows semantic versioning. It changes when chart structure or defaults change.
- appVersion: the version of the packaged application, such as nginx upstream version.
- description: short human-readable description.
- type: application or library. Library charts provide shared templates, not installable directly.
- keywords: searchable keywords for repositories.
- home: project homepage.
- sources: source code or documentation URLs.
- maintainers: list of maintainers for the chart.
- dependencies: external charts this chart depends on.
- annotations: arbitrary metadata for repository and tool integration.

EXAM TIP: version and appVersion are not interchangeable. Use version for chart package version, and appVersion for the application version inside the chart.

values.yaml

values.yaml contains default configuration for the chart.

Example:

```
replicaCount: 1
image:
  repository: nginx
  tag: 1.23.1
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
ingress:
  enabled: false
  hosts:
    - host: chart-example.local
      paths:
        - /
resources:
  limits:
    cpu: 100m
    memory: 128Mi
  requests:
    cpu: 100m
    memory: 128Mi
```

values.yaml defines default values that users can override with -f or --set.

charts/

The charts/ folder stores chart dependencies as packaged .tgz archives. Helm automatically downloads dependencies into this directory when you run helm dependency update.

templates/

The templates/ directory contains Go template files that generate Kubernetes manifests.

Common files:

- deployment.yaml: deployment resource template
- service.yaml: service resource template

- ingress.yaml: ingress resource template
- _helpers.tpl: partial templates and naming helpers
- NOTES.txt: post-install notes and instructions
- tests/test-connection.yaml: chart test pod

_helpers.tpl

_helpers.tpl contains named templates and partials.

Example:

```
define "nginx-deploy-chart.fullname"
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" -}}
end
```

Use helpers to avoid duplication and to centralize naming logic.

NOTES.txt

NOTES.txt is rendered after install or upgrade and printed to the user. It is useful for showing endpoint URLs, next steps, or credentials.

Example:

1. Get the application URL by running:

```
export POD_NAME=$(kubectl get pods --namespace {{ .Release.Namespace }} -l
"app.kubernetes.io/name={{ include "nginx-deploy-chart.name" . }}" -o
jsonpath='{.items[0].metadata.name}')
echo "Visit http://127.0.0.1:8080 to use your application"
```

COMMON MISTAKE: Putting critical resources in NOTES.txt. It should only be user guidance, not required YAML.

.helmignore

.helmignore lists files and directories to exclude from chart packages, similar to .gitignore.

Example:

```
.DS_Store
*.tgz
.git/
.gitignore
README.md
```

Chapter 4 summary

- A chart is a packaged Kubernetes application with metadata, defaults, templates, and optional dependencies.
- Chart.yaml, values.yaml, templates/, charts/, _helpers.tpl, and .helmignore are the main chart files.
- Chart.yaml fields document chart metadata.
- values.yaml defines defaults and supports overrides.
- NOTES.txt is for post-install user guidance.

Chapter 5 – Go Templating in Helm

Why Helm uses Go templates

Helm uses Go templates because they are fast, built into the Go standard library, and are expressive enough for Kubernetes manifest generation. They are not Jinja2 or Mustache because Go templates directly support the pipeline style and template functions Helm needs.

Template syntax and whitespace control

Basic syntax:

```
{{ .Values.image.repository }}
```

Whitespace controls:

- `{{ }}`: normal rendering
- `{{- }}`: trim whitespace to the left of the command
- `{{- -}}`: trim whitespace both before and after the command

Example:

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
...
{{- end }}
```

Whitespace control is critical in YAML templates to avoid invalid indentation.

Built-in objects

Helm exposes built-in objects inside templates.

- `.Release`: release metadata
- `.Chart`: chart metadata
- `.Values`: values passed to the chart
- `.Files`: files inside the chart package
- `.Capabilities`: cluster capability information
- `.Template`: template metadata

Example use:

```
metadata:
  name: {{ include "nginx-deploy-chart.fullname" . }}
labels:
  app.kubernetes.io/name: {{ .Chart.Name }}
  helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
```

.Release

Fields:

- `.Release.Name`: release name
- `.Release.Namespace`: release namespace
- `.Release.IsInstall`: true if installing
- `.Release.IsUpgrade`: true if upgrading
- `.Release.Revision`: release revision number
- `.Release.Service`: Helm service name (internal)

.Chart

Fields:

- `.Chart.Name`
- `.Chart.Version`
- `.Chart.AppVersion`
- `.Chart.Description`
- `.Chart.Type`

.Values

Contains values from `values.yaml`, CLI `--set`, files, and parent/sub-chart merging.

.Files

Use `.Files.Get` and `.Files.GetString` to embed chart files.

Example:

```
{{ .Files.Get "configmap.yaml" }}
```

.Capabilities

Fields include:

- `.Capabilities.KubeVersion.Version`
- `.Capabilities.APIVersions`
- `.Capabilities.KubeVersion.Major`
- `.Capabilities.KubeVersion.Minor`

Use it to conditionally render resources based on Kubernetes version.

.Template

Contains template engine metadata such as the template path.

Example:

```
# Generated by Helm template: {{ .Template.BasePath }}
```

Template functions

Common functions with examples:

- default: provide a fallback go image: `{{ .Values.image.repository | default "nginx" }}`
- required: fail if a value is missing go image: `{{ required "image.repository is required" .Values.image.repository }}`
- toYaml: convert objects to YAML go `{{ toYaml .Values.resources | indent 10 }}`
- fromYaml: parse YAML from a string
- indent: indent a block of text go `{{ toYaml .Values.config | indent 2 }}`
- nindent: indent and add newline before block go `{{ toYaml .Values.config | nindent 2 }}`
- quote: add double quotes
- upper, lower, trim
- include: render a named template and return string go `{{ include "nginx-deploy-chart.labels" . }}`
- tpl: evaluate a string as a template go `{{ tpl .Values.extraEnv . | indent 8 }}`
- lookup: query live cluster objects go `{{- $svc := lookup "v1" "Service" "default" "my-service" -}}`

COMMON MISTAKE: Using tpl for untrusted or arbitrary values. It can execute template expressions unexpectedly.

Pipelines

Helm uses pipelines to chain functions.

```
{{ .Values.image.tag | default "latest" | quote }}
```

This is equivalent to applying the last function to the result of the previous one.

Control structures

If/else:

```
{{- if .Values.ingress.enabled }}  
... ingress YAML ...  
{{- else }}  
# no ingress  
{{- end }}
```

Range:

```
{{- range .Values.env }}  
- name: {{ .name }}  
  value: {{ .value }}  
{{- end }}
```

With:

```
{{- with .Values.resources }}  
resources:  
  limits:  
    cpu: {{ .limits.cpu }}  
{{- end }}
```

Named templates with define, include, and template

Use define to declare reusable snippets.

Example:

```
define "nginx-deploy-chart.labels"  
app.kubernetes.io/name: {{ .Chart.Name }}  
app.kubernetes.io/instance: {{ .Release.Name }}  
end
```

Render with include:

```
labels:  
  {{ include "nginx-deploy-chart.labels" . | nindent 4 }}
```

include returns a rendered string. template is older and less recommended. Prefer include because it always returns a string and avoids subtle scoping issues.

The tpl function

tpl evaluates a string as a template in the current context.

Example use case:

extraEnv:

```
- name: PLACEHOLDER  
  value: "{{ .Release.Name }}"  
{{ tpl .Values.extraEnv . | nindent 8 }}
```

Danger: tpl can evaluate user-provided template expressions. Use it only when values intentionally contain Helm template syntax.

Common templating mistakes and gotchas

- forgetting `{{- end }}`
- invalid YAML indentation after templating
- using `.Values.foo.bar` when `foo` may be nil
- relying on latest tags or dynamic values in templates
- confusing `include` and `template`
- using `tpl` for untrusted strings

EXAM TIP: If you see a nil pointer evaluating interface error, inspect whether the value path exists and add default or required.

Chapter 5 summary

- Helm templates use Go templates with pipeline syntax.
- Built-in objects include `.Release`, `.Chart`, `.Values`, `.Files`, `.Capabilities`, and `.Template`.
- Use `default`, `required`, `toYaml`, `indent`, `nindent`, `include`, and `tpl` correctly.
- Control structures include `if`, `range`, and `with`.
- `_helpers.tpl` is the right place for reusable named templates.

Chapter 6 — Values: Configuration Management

The values hierarchy

Helm applies values in a strict precedence order, from lowest to highest:

1. Chart defaults: values.yaml
2. Sub-chart values
3. Parent chart values
4. User-supplied values file(s) with -f
5. --set flags

This means a value set with --set always overrides values in values.yaml and -f files.

--set vs -f vs --set-string vs --set-file vs --set-json

Option	Purpose	Use case
-f custom.yaml	merge values from a YAML file	environment-specific configuration
--set key=value	override a scalar value	simple CLI overrides
--set-string key=value	force string even if numeric	preserve leading zeros or phone numbers
--set-file key=path	load file contents into a string value	certificates, scripts
--set-json key='{ "a": 1 }'	set structured JSON value	complex nested data without file

Examples:

```
helm install myapp ./chart -f prod-values.yaml --set replicaCount=3
helm upgrade myapp ./chart --set-string image.tag=2025.04.01
helm install myapp ./chart --set-file tls.crt=./cert.pem
```

COMMON MISTAKE: Using --set for complex nested structures. Prefer -f for YAML files and --set-json for structured data.

Merging behavior: deep merge vs shallow merge

Helm performs a deep merge of YAML values by default. This means nested maps are merged recursively.

Example:
values.yaml:

```
image:
  repository: nginx
  tag: 1.23.1
```

custom.yaml:

```
image:
  tag: 1.24.0
helm install myapp ./chart -f custom.yaml
```

Result:

```
image:
  repository: nginx
  tag: 1.24.0
```

Shallow merge occurs when replacing entire maps with `--set`. For nested map overrides, use explicit full path or `-f`.

Referencing values across sub-charts

Sub-chart values are normally isolated. To pass values from a parent chart to sub-charts, use ``global``:

```
```yaml
global:
 redis:
 password: secret
```

Then in the sub-chart template:

```
```go
{{ .Values.global.redis.password }}
```

Parent chart values can also be passed by naming the sub-chart key.

Validating values with `values.schema.json``

Helm supports JSON Schema validation for chart values.

Example `values.schema.json``:

```
```json
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "type": "object",
 "properties": {
```

```
"replicaCount": {
 "type": "integer",
 "minimum": 1
}
,
"required": ["replicaCount"]
}
```

When installed, Helm validates values against this schema and fails early if the user provides invalid data.

## The required function as a guard

Use required in templates to fail if a value is missing.

Example:

```
name: {{ required "The service name is required" .Values.service.name }}
```

This is especially useful for values that should never be omitted.

## Secrets in values

**Never commit secrets in values.yaml.** Helm values are stored in release metadata, which may be readable by cluster users.

Secure alternatives:

- Mozilla SOPS with Helm Secrets plugin
- HashiCorp Vault and External Secrets Operator
- Kubernetes Secrets created outside Helm

**COMMON MISTAKE:** Storing passwords in values.yaml or using `--set password=mypassword` in CI. Use secret mechanisms instead.

## Chapter 6 summary



- Helm values follow strict precedence: chart defaults, sub-chart, parent chart, -f, --set.
- Use -f for files and --set for simple CLI overrides.
- --set-string, --set-file, and --set-json cover string, file, and structured values.
- values.schema.json validates values.
- Never store secrets in chart values directly.

## PART 3: HELM CLI — COMPLETE COMMAND REFERENCE

### Chapter 7 — Repository Management

#### What a Helm repository is

A Helm repository is an HTTP server that hosts packaged charts and an index file (index.yaml). The index lists chart names, versions, and metadata.

Repositories are analogous to apt or npm registries.

#### helm repo add

Add a repository:

```
helm repo add bitnami https://charts.bitnami.com/bitnami
```

For private repos with basic auth:

```
helm repo add secure https://myrepo.example.com/charts --username user --password pass
```

#### helm repo list, helm repo update, and helm repo remove

List repos:

```
helm repo list
```

Update local repository cache:

helm repo update

Remove a repository:

helm repo remove bitnami

## helm search repo vs helm search hub

helm search repo searches charts in locally added repositories:

helm search repo nginx

helm search hub searches Artifact Hub, the public Helm chart index:

helm search hub nginx

**EXAM TIP:** helm search repo works offline after helm repo update. helm search hub requires internet access and queries Artifact Hub.

## OCI registries

Helm supports OCI-based storage like Docker registries. OCI registries store charts as OCI artifacts instead of index-based repos.

Login:

helm registry login ghcr.io

Push chart:

helm push mychart-0.1.0.tgz oci://ghcr.io/my-org/charts

Pull chart:

helm pull oci://ghcr.io/my-org/charts/mychart --version [0.1.0](#)

OCI is the future of Helm distribution because it unifies chart storage with container artifact registries.

## Popular repositories

- Bitnami: <https://charts.bitnami.com/bitnami>
- Artifact Hub: search index for Helm charts
- Stable repository: deprecated, do not rely on it

## Setting up a private chart repository

A private chart repo can be a static file server, S3 bucket, or GitHub Pages hosting an `index.yaml` and chart archives.

Generate index:

```
helm repo index ./ --url https://example.com/charts
```

Upload `.tgz` packages and `index.yaml` to your hosting location.

**COMMON MISTAKE:** Skipping `helm repo index` after adding charts. The index file must list every chart version.

## Chapter 7 summary

- Helm repositories serve packaged charts through HTTP.
- Use `helm repo add`, `helm repo list`, `helm repo update`, and `helm repo remove`.
- `helm search repo` searches local repos, while `helm search hub` queries Artifact Hub.
- OCI registries are a modern distribution method for Helm charts.

## Chapter 8 — Installing and Managing Releases

### What a Release is

A Helm release is a named, versioned instance of a chart installed into a cluster. Each release has history revisions and can be upgraded or rolled back.

### helm install syntax and flags

Basic install:

```
helm install my-release ./nginx-deploy-chart
```

Common flags:

- --namespace: target namespace
- --create-namespace: create namespace if it does not exist
- --values / -f: supply values file(s)
- --set: override values on CLI
- --dry-run: render templates without installing
- --debug: show detailed output
- --wait: wait for resources to become ready
- --timeout: adjust wait timeout
- --atomic: roll back on failure
- --generate-name: generate a release name automatically

Example:

```
helm install myapp ./chart --namespace production --create-namespace -f prod-values.yaml --set replicaCount=3 --wait --timeout 5m --atomic
```

**EXAM TIP:** --atomic ensures the release is rolled back automatically if installation fails.

## helm list

List releases in the current namespace:

```
helm list
```

List all namespaces:

```
helm list -A
```

List a specific namespace:

```
helm list -n dev
```

## helm status

Show release details:

```
helm status my-release
```

It reports release status, namespace, revision, notes, and deployed resources.

## helm get subcommands

helm get all <release>: export all release information

helm get values <release>: show user-supplied and computed values

helm get values <release> --all: show all values including defaults

helm get manifest <release>: show rendered manifests

helm get hooks <release>: show hook objects

helm get notes <release>: show rendered NOTES.txt

Example:

```
helm get manifest my-release
```

## helm uninstall

Remove a release:

```
helm uninstall my-release
```

By default, Helm deletes Kubernetes resources and release metadata.

Keep history while removing resources:

```
helm uninstall my-release --keep-history
```

## Release naming conventions and best practices

- use meaningful, short names
- avoid uppercase and special characters
- keep names under 53 characters for Kubernetes resources
- use separate release names for different environments

# helm template

Render manifests locally without a cluster:

```
helm template my-release ./chart -f values.yaml
```

This is critical for CI/CD validation and debugging.

## Chapter 8 summary

- A release is a chart installation instance in the cluster.
- `helm install` has flags for namespace, values, waiting, and failure handling.
- Use `helm status`, `helm get`, and `helm list` to inspect releases.
- `helm uninstall --keep-history` removes resources but preserves release history.
- `helm template` renders manifests without contacting the cluster.

## Chapter 9 — Upgrading, Rolling Back, and History

### helm upgrade and flags

Upgrade a release:

```
helm upgrade my-release ./chart -f values.yaml --set image.tag=1.24.0
```

Important flags:

- `--install`: install if the release does not exist
- `--reset-values`: reset values to chart defaults, dropping previous user values
- `--reuse-values`: keep existing values and merge new ones
- `--force`: force resource replacement if update fails
- `--cleanup-on-fail`: delete new resources after failed install
- `--atomic`: rollback if upgrade fails

`--reset-values` vs `--reuse-values`:

- `--reset-values`: use chart defaults plus current overrides; discard prior values.
- `--reuse-values`: keep release's previous values and merge new overrides.

**COMMON MISTAKE:** Using `--reuse-values` blindly. It can keep stale values or sensitive values from prior installs.

## helm history

View revision history:

```
helm history my-release
```

Interpretation:

- REVISION: release revision number
- UPDATED: timestamp
- STATUS: deployed, superseded, failed, pending-upgrade, pending-install, etc.
- CHART: chart version
- APP VERSION: appVersion from chart

## helm rollback

Rollback to a previous revision:

```
helm rollback my-release 2
```

Rollback restores release state to the selected revision. It does not guarantee zero-downtime, but it restores resource manifests from that revision.

## Failed upgrade scenarios

If a release is stuck in pending-upgrade, the cluster or API server may have timed out. Fix it by:

- waiting for the operation to finish
- checking `helm get manifest` and related resource events
- using `helm rollback --cleanup-on-fail --atomic` in the future

## Zero-downtime upgrade strategy with Helm

Use these patterns:

- readiness and liveness probes
- --wait and --timeout
- --atomic on upgrades
- maxUnavailable set to 0 or a small value in rolling updates
- helm diff plugin before the upgrade

## Chapter 9 summary

- helm upgrade changes a release and can optionally install with --install.
- --reset-values drops prior values, --reuse-values preserves them.
- helm history shows revision history and statuses.
- helm rollback restores a prior revision.
- Use --atomic to avoid broken upgrades.

## Chapter 10 – Chart Dependencies

### What sub-charts are and why they exist

Sub-charts are nested charts included in a parent chart. Use them when your application depends on other Helm packages.

Example: a web app chart can depend on a Redis sub-chart.

### Defining dependencies in Chart.yaml

Example:

```
dependencies:
- name: redis
 version: 19.10.1
 repository: https://charts.bitnami.com/bitnami
 condition: redis.enabled
tags:
- database
```

### helm dependency update



Downloads dependencies and places them into charts/:

```
helm dependency update ./nginx-deploy-chart
```

## **helm dependency build**

Rebuilds dependencies from Chart.lock without fetching versions from repositories.

```
helm dependency build ./nginx-deploy-chart
```

## **helm dependency list**

Inspect dependencies:

```
helm dependency list ./nginx-deploy-chart
```

## **Chart.lock vs Chart.yaml**

- Chart.yaml: declares dependency ranges and repository references.
- Chart.lock: pins exact dependency versions after helm dependency update.

Chart.lock is like package-lock.json for Helm. Commit it to keep builds reproducible.

## **Condition and tags**

Enable/disable sub-charts with values.

Example:

```
redis:
 enabled: false
```

With tags:

```
tags:
 database: true
```

Use condition and tags to control which sub-charts are enabled.

## Global values for parent and sub-charts

Use global values to share configuration across all charts:

global:

storageClass: fast

Sub-charts can reference `.Values.global.storageClass`.

## When to use sub-charts vs separate releases

Use sub-charts when:

- the dependency is tightly coupled
- you want a single install command
- configuration belongs in the parent chart

Use separate releases when:

- the dependency is independently managed
- you need separate lifecycle or upgrades
- teams own the dependency separately

## Chapter 10 summary

- Dependencies are declared in `Chart.yaml` and downloaded to `charts/`.
- `helm dependency update` fetches dependencies; `helm dependency build` rebuilds from `Chart.lock`.
- `Chart.lock` should be committed for reproducibility.
- Use conditions, tags, and global values to control sub-charts.

## Chapter 11 – Testing, Linting, and Validation

### helm lint

Lint a chart to catch common issues:

```
helm lint ./nginx-deploy-chart
```

It checks chart structure, YAML validity, missing required fields, and template validation.

## helm template for validation

Render templates without a cluster:

```
helm template my-release ./nginx-deploy-chart -f values.yaml
```

Use this to validate YAML syntax and debug templates before installing.

## helm test

Helm can run test hooks after install or upgrade.

Example test pod annotation:

```
apiVersion: v1
kind: Pod
metadata:
 name: "{{ include \"nginx-deploy-chart.fullname\" . }}-test"
 annotations:
 "helm.sh/hook": test
spec:
 containers:
 - name: curl
 image: curlimages/curl:7.85.0
 command: ["curl", "-f", "http://localhost"]
 restartPolicy: Never
```

Run tests:

```
helm test my-release
```

## --dry-run vs helm template

- helm template only renders templates locally; it does not contact the Kubernetes API.
- helm install --dry-run renders templates and simulates an install, including some validation, but still does not create resources.

**EXAM TIP:** Use `helm template` for offline validation, and `helm install --dry-run` for a closer simulation of install behavior.

## Schema validation tools

Validate generated manifests with tools like `kubeval` and `kubeconform`:

```
helm template my-release ./chart | kubeval --strict
```

## CI/CD pipeline integration

A recommended order:

1. `helm lint`
2. `helm template`
3. `kubeval` or `kubeconform`
4. `helm diff` or `helm upgrade --dry-run`
5. actual `helm upgrade`

**COMMON MISTAKE:** Skipping local rendering before install. Always render templates to catch syntax and template errors early.

## Chapter 11 summary

- Use `helm lint` to validate charts.
- Use `helm template` to render manifests locally.
- Use `helm test` with hook pods for release tests.
- `--dry-run` simulates install/upgrade while still validating.
- Add `kubeval` or `kubeconform` to CI pipelines.

## Chapter 12 — Packaging and Distribution

### helm package

Package a chart into a .tgz archive:

```
helm package ./nginx-deploy-chart
```

The generated file is nginx-deploy-chart-0.1.0.tgz.

## helm verify

If the chart is signed, verify it with:

```
helm verify nginx-deploy-chart-0.1.0.tgz
```

This checks the chart provenance signature.

## helm repo index

Generate an index file for a chart repository:

```
helm repo index . --url https://example.com/charts
```

Place the .tgz and index.yaml together on a web server.

## Hosting chart repositories

Common hosting options:

- GitHub Pages
- Amazon S3
- Nexus
- Harbor

For GitHub Pages:

1. Package charts to docs/
2. Run `helm repo index docs/ --url https://myuser.github.io/repo`
3. Publish docs/ as GitHub Pages branch

# Pushing to OCI registries

OCI chart flow:

```
helm chart save ./nginx-deploy-chart oci://ghcr.io/my-org/nginx
```

```
helm chart push oci://ghcr.io/my-org/nginx:0.1.0
```

Pull from OCI:

```
helm chart pull oci://ghcr.io/my-org/nginx:0.1.0
```

```
helm chart export oci://ghcr.io/my-org/nginx:0.1.0
```

## Chapter 12 summary

- helm package creates chart archives.
- helm verify checks signed charts.
- helm repo index generates repository indexes.
- Charts can be hosted on GitHub Pages, S3, Nexus, or Harbor.
- OCI registries are an alternative distribution mechanism.

# PART 4: ADVANCED HELM

## Chapter 13 – Helm Hooks

### What hooks are and why they exist

Hooks are special resources that Helm runs at prescribed lifecycle events. They exist to handle tasks that must happen before or after normal chart installs, upgrades, deletes, rollbacks, or tests.

Common hook use cases:

- database migrations before application startup
- smoke tests after deployment
- cleanup jobs before uninstall

### All hook types

Hook type	When it runs
pre-install	before install resources are created
post-install	after install succeeds
pre-upgrade	before upgrade resources are applied
post-upgrade	after upgrade succeeds
pre-delete	before release deletion
post-delete	after release deletion
pre-rollback	before rollback starts
post-rollback	after rollback completes
test	when helm test is run

## Hook annotations

Define hooks using annotations in metadata.

Example job as a pre-upgrade hook:

```
apiVersion: batch/v1
kind: Job
metadata:
 name: "{{ include "mychart.fullname" . }}-migrate"
 annotations:
 "helm.sh/hook": pre-upgrade
 "helm.sh/hook-weight": "0"
 "helm.sh/hook-delete-policy": hook-succeeded
spec:
 template:
 spec:
 containers:
 - name: migrate
 image: alpine
 command: ["sh", "-c", "echo migrate"]
 restartPolicy: Never
```

helm.sh/hook-weight controls execution order when multiple hooks of the same type exist.

## hook-delete-policy

Possible values:

- before-hook-creation
- hook-succeeded
- hook-failed

Example:

annotations:

"helm.sh/hook-delete-policy": hook-succeeded,hook-failed

This ensures the hook resource is cleaned after completion.

## Real use cases

- pre-install: create database schema or namespace prerequisites
- post-install: print connection details or run smoke tests
- pre-delete: backup data before removing resources
- test: verify service availability after install

## Hook failure modes

When a hook fails, Helm marks the release accordingly, and the release can remain in a pending state. Use `helm get hooks` to inspect hook resources.

**COMMON MISTAKE:** Using hooks for regular application resources. Hooks should only be used for lifecycle jobs or tasks, not primary service resources.

## Chapter 13 summary

- Hooks run at lifecycle events like install, upgrade, delete, rollback, and test.
- Use `helm.sh/hook`, `helm.sh/hook-weight`, and `helm.sh/hook-delete-policy` annotations.
- Hooks are ideal for migrations, smoke tests, backups, and cleanup tasks.



# Chapter 14 – Helm Secrets & Security

## How Helm stores release state as Kubernetes Secrets

By default, Helm stores release metadata in Kubernetes Secrets in the release namespace. The secret contains base64-encoded YAML describing the chart, values, manifest, and hooks.

This is not encrypted, only encoded.

## Finding Helm release secrets

List secrets:

```
kubectl get secrets -n production
```

Release secrets are named like `sh.helm.release.v1.<release>.v<revision>`.

Decode a release secret:

```
kubectl get secret sh.helm.release.v1.my-release.v1 -n production -o jsonpath="{.data.release}" |
base64 --decode | base64 --decode | gunzip -c
```

## Security implications

Anyone with access to these secrets can read release data, including config values. Protect them with proper RBAC and namespace restrictions.

## RBAC for Helm

A user or service account needs permissions to:

- create/update/delete resources in the target namespace
- read created resources
- create Secrets or ConfigMaps for release storage

Minimal RBAC example for namespace install:

apiVersion: rbac.authorization.k8s.io/v1

kind: Role

metadata:

name: helm-deployer

namespace: production

rules:

- apiGroups: ["\*"]

resources: ["\*"]

verbs: ["\*"]

For a more secure setup, narrow permissions to the actual resources needed.

## Supply chain security

Helm supports chart signing and provenance.

Sign a chart:

```
helm package ./chart --sign --key mykey --keyring ~/.gnupg/secring.gpg
```

Verify a chart:

```
helm verify mychart-0.1.0.tgz
```

This protects against tampered charts in repositories.

## Secrets management patterns

Safe approaches:

- Helm Secrets plugin with Mozilla SOPS
- HashiCorp Vault Agent or External Secrets Operator
- Kubernetes Secrets managed separately from Helm values

Never pass secrets directly on the command line:

```
helm install myapp ./chart --set password=mypassword
```

**COMMON MISTAKE:** Exposing secrets through shell history and audit logs. Use files or secret backends.

## Chapter 14 summary

- Helm stores release state in Secrets by default.
- Release secrets are not encrypted; they are base64-encoded.
- Use RBAC to limit who can read release secrets.
- Sign charts and use provenance for supply chain security.
- Avoid `--set password=` in production.

## Chapter 15 – Helm in CI/CD Pipelines

### Helm in GitOps workflows

GitOps tools such as ArgoCD and Flux use Helm charts as application sources. They do not necessarily call `helm install` directly, but they use Helm libraries or controllers to render charts and deploy releases.

ArgoCD can sync a Helm chart from a Git repository or chart repository and manage lifecycle automatically.

### Recommended CI/CD pipeline

A production-safe Helm pipeline:

1. `helm lint ./chart`
2. `helm template my-release ./chart -f values.yaml`
3. `kubeval` or `kubeconform` on rendered YAML
4. `helm diff upgrade my-release ./chart -f values.yaml`
5. `helm upgrade --install my-release ./chart -f values.yaml --atomic --wait`

This sequence ensures chart correctness, schema validation, and safe rollout.

### helm diff plugin

The helm diff plugin shows the difference between installed release manifests and the rendered chart.

```
helm diff upgrade my-release ./chart -f values.yaml
```

This is essential before production upgrades to understand what will change.

## helm secrets plugin

The Helm Secrets plugin integrates with Mozilla SOPS to encrypt values files.

Example:

```
helm secrets enc values-prod.yaml
```

```
helm secrets dec values-prod.yaml
```

This protects secrets in Git while allowing Helm to decrypt them at deploy time.

## Environment promotion pattern

Use separate values files for each environment:

- values-dev.yaml
- values-staging.yaml
- values-prod.yaml

Promote with different values files and release names.

## Helmfile

Helmfile is a declarative tool for managing multiple Helm releases in one YAML file. It is different from running multiple helm install commands because it expresses all release definitions in a single source-of-truth file.

**EXAM TIP:** Understand Helmfile conceptually, but focus on native Helm commands unless the

question specifically mentions Helmfile or GitOps tools.

## Chapter 15 summary

- Use Helm in GitOps and CI/CD with lint, template, validation, diff, and atomic upgrades.
- helm diff is critical before production upgrades.
- Use helm secrets with SOPS for encrypted values.
- Helmfile manages multiple releases declaratively.

## Chapter 16 – Chart Best Practices (Official Helm Guidelines)

### Naming conventions

- chart names should be lowercase and hyphen-separated
- release names should be meaningful and concise
- use {{ include "chart.fullname" . }} for resource names to avoid collisions

### Image tag best practices

- never use latest in production default values
- use explicit tags or digests
- keep appVersion in sync with the image tag when possible

### Resource naming

Use helper templates for full names:

```
define "nginx-deploy-chart.fullname"
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" -}}
end
```

### Labels and selectors

Always include standard labels:

- `helm.sh/chart`
- `app.kubernetes.io/name`
- `app.kubernetes.io/instance`
- `app.kubernetes.io/managed-by`

These labels improve discovery and tooling compatibility.

## Resource limits and requests

Always define CPU and memory requests and limits in production charts.

Example:

```
resources:
 requests:
 cpu: 100m
 memory: 128Mi
 limits:
 cpu: 250m
 memory: 256Mi
```

## Liveness and readiness probes

Include both probes to support stable rollouts and health checks.

## Horizontal Pod Autoscaler

Include HPA as an optional resource, disabled by default.

## PodDisruptionBudget

Include a PodDisruptionBudget (PDB) for production charts to protect availability during voluntary disruptions.

## NetworkPolicy

Make NetworkPolicy opt-in to allow charts to work in clusters without network policies while still supporting secure deployments.

## ServiceAccount

Create a dedicated ServiceAccount for the chart when the workload needs specific permissions.

## Chapter 16 summary

- Follow chart naming conventions and standard labels.
- Never use latest in production defaults.
- Always define resource requests/limits, probes, and optional production constructs like PDB.
- Use helper templates for consistent naming.

# PART 5: REAL-WORLD WORKFLOWS

## Chapter 17 — The nginx Deployment Workflow (From Source Material)

We will walk through the workflow from raw kubectl manifests into a Helm chart.

### Step 1: Create deployment manifest

Source command:

```
kubectl create deploy nginx --image=nginx:alpine --port=80 -o yaml > deploy.yaml
```

Explanation:

- `kubectl create deploy nginx`: creates a Deployment resource.
- `--image=nginx:alpine`: sets the container image.
- `--port=80`: annotates container port metadata.
- `-o yaml`: outputs YAML instead of applying it.

**NOTE:** `kubectl create ... -o yaml` does not apply the manifest unless you add `--dry-run=client` or `--dry-run=server`.

Live example with dry-run:

```
kubectl create deploy nginx --image=nginx:alpine --port=80 --dry-run=client -o yaml > deploy.yaml
```

Review the generated YAML and identify values to parameterize.

## Step 2: Create service manifest

Source command:

```
kubectl expose deploy nginx --target-port=80 -o yaml --dry-run=client > deploy-expose-service.yaml
```

Explanation:

- `kubectl expose deploy nginx`: creates a Service resource for the deployment.
- `--target-port=80`: forwards service traffic to container port 80.
- `-o yaml --dry-run=client`: output the manifest without applying.

Service type selection:

- `ClusterIP`: internal cluster access (default)
- `NodePort`: exposes service on a node port
- `LoadBalancer`: requests cloud load balancer

For exam and production, use `ClusterIP` for internal services and `LoadBalancer` only when externally reachable.

## Step 3: Create Helm chart

Source command:

```
helm create nginx-deploy-chart
```



This generates a starter chart with sample templates and files. Modify or remove the generated files to match your workload.

Expected generated files:

- Chart.yaml
- values.yaml
- templates/deployment.yaml
- templates/service.yaml
- optional ingress, notes, helper templates

## Convert raw YAML into Helm templates

Take deploy.yaml and deploy-expose-service.yaml and replace hard-coded values with Helm template expressions.

Example templates/deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: {{ include "nginx-deploy-chart.fullname" . }}
 labels:
 app.kubernetes.io/name: {{ include "nginx-deploy-chart.name" . }}
 app.kubernetes.io/instance: {{ .Release.Name }}
spec:
 replicas: {{ .Values.replicaCount }}
 selector:
 matchLabels:
 app.kubernetes.io/name: {{ include "nginx-deploy-chart.name" . }}
 app.kubernetes.io/instance: {{ .Release.Name }}
 template:
 metadata:
 labels:
 app.kubernetes.io/name: {{ include "nginx-deploy-chart.name" . }}
 app.kubernetes.io/instance: {{ .Release.Name }}
 spec:
 containers:
 - name: nginx
 image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

```
ports:
- containerPort: {{ .Values.containerPort }}
```

Example templates/service.yaml:

```
apiVersion: v1
kind: Service
metadata:
 name: {{ include "nginx-deploy-chart.fullname" . }}
 labels:
 app.kubernetes.io/name: {{ include "nginx-deploy-chart.name" . }}
 app.kubernetes.io/instance: {{ .Release.Name }}
spec:
 type: {{ .Values.service.type }}
 ports:
 - port: {{ .Values.service.port }}
 targetPort: {{ .Values.service.targetPort }}
 selector:
 app.kubernetes.io/name: {{ include "nginx-deploy-chart.name" . }}
 app.kubernetes.io/instance: {{ .Release.Name }}
```

Example values.yaml:

```
replicaCount: 2
image:
 repository: nginx
 tag: alpine
containerPort: 80
service:
 type: ClusterIP
 port: 80
 targetPort: 80
```

Example Chart.yaml:

```
apiVersion: v2
name: nginx-deploy-chart
description: A simple Helm chart for nginx
type: application
version: 0.1.0
appVersion: 1.23.1
```

Modify generated templates as needed and delete unused resources like templates/ingress.yaml if not required.

# Install, upgrade, rollback the release

Install the chart:

```
helm install nginx-demo ./nginx-deploy-chart --namespace web --create-namespace
```

Upgrade with a new tag:

```
helm upgrade nginx-demo ./nginx-deploy-chart --set image.tag=1.23.8
```

Rollback if needed:

```
helm rollback nginx-demo 1
```

Inspect release status:

```
helm status nginx-demo
```

```
helm get manifest nginx-demo
```

```
helm get values nginx-demo
```

**EXAM TIP:** When converting raw manifests to Helm, always parameterize the values that differ across environments and keep the chart generic.

## Chapter 17 summary

- Use `kubectl create ... -o yaml --dry-run=client` to generate base manifests.
- Convert manifests into Helm templates with `{{ .Values.* }}`.
- Keep values in `values.yaml` and override with `-f` or `--set`.
- Use `helm install`, `helm upgrade`, and `helm rollback` to manage the release.

## Chapter 18 – Troubleshooting Helm

### pending-install or pending-upgrade

Causes:

- cluster timeout during install/upgrade
- hook failure

- API server connectivity issues

Fix:

- inspect events with `kubectl get events -n <ns>`
- inspect release with `helm status <release>` and `helm history <release>`
- use `helm rollback <release> <revision>` if needed

## cannot re-use a name that is still in use

This occurs when a failed install left a release name in use with no deployed revision.

Fix:

- `helm uninstall <release>` if the release exists
- `helm install <release> ./chart --replace` to reuse the name in Helm v3 if the release exists but no deployed resources

## release has no deployed releases during rollback

Meaning:

- there is no previous successful deployed revision to rollback to

Recovery:

- inspect `helm history <release>`
- if there is no deployed revision, install the chart fresh or use a known good revision from history

## Failed hooks blocking releases

Use:

`helm get hooks <release>`

```
kubectl describe job <hook-job> -n <namespace>
```

```
kubectl logs job/<hook-job> -n <namespace>
```

Clean up failed hooks with `kubectl delete job <name> -n <namespace>` if it is safe, then retry the Helm operation.

## Inspect deployed manifest

Use:

```
helm get manifest <release>
```

```
helm template <release> ./chart -f values.yaml
```

## Deep template debugging

Use:

```
helm template my-release ./chart --debug
```

```
helm install my-release ./chart --dry-run --debug
```

## Compare desired vs actual state

Use the diff plugin:

```
helm diff upgrade my-release ./chart -f values.yaml
```

## Chapter 18 summary

- pending-install and pending-upgrade indicate stuck operations.
- cannot re-use a name usually means a failed install or incomplete release.
- release has no deployed releases means rollback is impossible.
- Use `helm get hooks`, `kubectl describe`, and `helm diff` to troubleshoot.

# PART 6: EXAM PREPARATION

## Chapter 19 — CKA / CKAD / KCNA Exam Questions

### CONCEPTUAL / WHY QUESTIONS

1. Why was Helm created? What specific problem does it solve that kubectl apply -f does not?

Helm was created to provide package management for Kubernetes applications. While kubectl apply -f applies raw YAML resources, Helm enables reusable charts, parameterized configuration, release versioning, lifecycle management, and dependency handling. kubectl apply -f does not track a release history, manage upgrades and rollbacks in a structured way, or package resources as reusable software.

2. Explain the difference between a Chart, a Release, and a Revision.

- **Chart:** a reusable package of Kubernetes manifests, metadata, and default values.
- **Release:** a specific installation of a chart into a cluster, identified by a name and namespace.
- **Revision:** a version of a release after an install or upgrade. Each change increments the revision number.

3. Why was Tiller removed in Helm v3? What security vulnerabilities did it introduce?

Tiller was removed because it introduced a server-side component with cluster-wide privileges. Security issues included:

- Tiller acting with elevated permissions independent of the user
- poor RBAC control and auditability
- a central point for privilege escalation By removing Tiller, Helm v3 uses the user's kubeconfig credentials directly.

4. How is Helm different from Kustomize? When would you choose one over the other?

Helm is a templating and package manager. Kustomize is a resource customization tool.

- Choose **Helm** when you need reusable packages, dynamic values, and release lifecycle management.
- Choose **Kustomize** when you want to patch existing manifests without templates and keep overlays simple.

**EXAM TIP:** Helm is better for packaged applications; Kustomize is better for environment-specific overlays on top of static manifests.

5. Why does Helm store release state as Kubernetes Secrets by default? What are the

alternatives and their tradeoffs?

Helm stores release state as Secrets because it keeps metadata close to the release namespace and allows secure access control through Kubernetes RBAC. Alternatives are ConfigMaps, memory, and SQL.

- **ConfigMaps**: easier to inspect, less secure.
- **Memory**: non-persistent, only for testing.
- **SQL**: external storage, more complex.

6. What is the difference between appVersion and version in Chart.yaml?

- **version**: the chart package version. It changes when the chart itself changes.
- **appVersion**: the version of the application being deployed, such as nginx version.

7. Why should you never use latest as an image tag in a Helm chart's default values.yaml?

latest is not immutable and can lead to unpredictable deployments and inability to reproduce versions. It breaks deterministic releases and is unsafe for production.

8. What is the difference between helm template and helm install --dry-run?

- **helm template** renders templates locally without any interaction with the cluster.
- **helm install --dry-run** simulates an install and validates resource creation logic, but still does not create resources.

9. Why does Helm use Go templates instead of a simpler format?

Go templates are powerful, integrated with Helm's Go codebase, and support functions, pipelines, and control structures. They provide the flexibility required for Kubernetes YAML generation without adding another external engine.

10. What problem do Helm hooks solve? Give three real-world use cases.

Hooks solve lifecycle tasks that must occur before or after chart operations. Use cases:

- run database migrations before an upgrade (pre-upgrade)
- run smoke tests after install (post-install or test)
- backup data before uninstall (pre-delete)

11. Why is `--reuse-values` during upgrade potentially dangerous?

Because it retains old values from the release and merges new overrides, which can unintentionally preserve stale or sensitive settings. This can lead to configuration drift and unexpected behavior.

12. What is the purpose of `_helpers.tpl` and why is it a best practice?

`_helpers.tpl` centralizes reusable named templates such as labels, names, and selectors. It avoids duplication and keeps templates consistent across resources.

13. Explain the values precedence hierarchy in Helm from lowest to highest priority.

14. Chart defaults from `values.yaml`

15. Sub-chart values

16. Parent chart values

17. user-supplied values files with `-f`

18. `--set` values

19. Why is helm rollback not always safe? What can go wrong?

Rollback restores previous manifest definitions, but it does not restore deleted data or handle stateful application changes gracefully. A rollback may also reintroduce old configuration that is incompatible with current cluster state.

15. What is the difference between `helm dependency update` and `helm dependency build`?

- `helm dependency update` downloads dependencies from repositories and updates `Chart.lock`.
- `helm dependency build` downloads dependencies based on `Chart.lock` and recreates charts/without changing the lock file.

## ARCHITECTURE / INTERNALS QUESTIONS

16. How does Helm v3 communicate with the Kubernetes API server? Draw/describe the request flow.

Flow:

1. Helm reads `kubeconfig` and context.
2. Helm renders chart templates locally.
3. Helm contacts the Kubernetes API using `client-go`.
4. Helm submits standard Kubernetes requests (create, patch, delete).



5. Helm stores release state in Secrets or ConfigMaps.

6. Where does Helm store release state? How would you find and read a Helm release secret with kubectl?

It stores release state in Kubernetes Secrets by default. To read it:

```
kubectl get secret -n production
```

```
kubectl get secret sh.helm.release.v1.my-release.v1 -n production -o jsonpath="{.data.release}" |
base64 --decode | base64 --decode | gunzip -c
```

18. What is the structure of a Helm release secret? What information does it contain?

A release secret contains base64-encoded data with fields such as:

- release: the compressed release payload
- metadata with name, namespace, labels The payload includes chart metadata, values, manifest, hooks, and timestamps.

19. How does Helm handle RBAC – what Kubernetes permissions does a user need to install a release?

Helm uses the user's kubeconfig credentials. The user needs permissions to create, update, and delete the target resources and to create Secret/ConfigMap objects for release storage in the namespace.

20. Explain how helm search hub works differently from helm search repo. What is Artifact Hub?

- helm search repo: searches charts in locally configured Helm repositories.
- helm search hub: searches Artifact Hub, which is a public aggregator of Helm charts and other cloud-native packages.

Artifact Hub is a web-based discovery service for Helm charts, OLM operators, Falco rules, and more.

21. What is an OCI registry in the context of Helm? How does it differ from a traditional Helm repository?

An OCI registry stores Helm charts as OCI artifacts using the same protocol as container registries. Traditional Helm repos use index.yaml and tarball hosting. OCI registries support OCI authentication and more standardized storage.

22. Explain the Chart.lock file. When is it generated and when should you commit it?

Chart.lock is generated by helm dependency update. It pins exact dependency versions. Commit it to version control to ensure repeatable builds.

23. How does Helm resolve template rendering order within a chart?

Helm renders all templates and then sorts resources by Kubernetes kind and metadata order. Hooks are run according to hook type and weight before or after normal resources.

24. What happens internally when you run helm upgrade --atomic?

Helm performs the upgrade and monitors the release. If the upgrade fails or resources do not become ready in time, Helm rolls back to the prior revision automatically.

25. How does helm rollback work at the Kubernetes object level?

Helm re-renders the prior revision's manifest and applies it to the cluster. It treats the rollback as a new release revision and updates resources accordingly.

## TEMPLATING QUESTIONS

26. What is the difference between include and template in Helm? Which should you prefer and why?

include returns a rendered string from a named template. template renders a named template directly and is older syntax. Prefer include because it is clearer and easier to compose with pipes.

27. Explain the difference between {{ }}, {{- }}, and {{- -}} in Go templates.

- {{ }}: preserves surrounding whitespace.
- {{- }}: trims whitespace to the left of the command.
- {{- -}}: trims whitespace both before and after the command.

28. What does nindent do? Why is it used instead of indent?

nindent adds a newline then indents the rendered block. It is useful when generating YAML blocks that require a new line before indentation.

Example:

```
{{ toYaml .Values.resources | nindent 8 }}
```

29. Write a template snippet that makes a value required and fails with a custom message if not set.

```
image: {{ required "image.repository must be set" .Values.image.repository }}
```

30. Explain the .Release object. List and explain all its fields.

.Release fields include:

- .Release.Name: name of the Helm release
- .Release.Namespace: target namespace
- .Release.IsInstall: true during install
- .Release.IsUpgrade: true during upgrade
- .Release.IsRollback: true during rollback
- .Release.Revision: current release revision number
- .Release.Service: Helm service name
- .Release.IsUpgrade and .Release.IsInstall help conditionally render templates

31. Explain the .Capabilities object. Give a use case for checking Kubernetes API version in a template.

.Capabilities exposes the cluster's supported API versions and Kubernetes version. Use it to render resources conditionally:

```
{{- if semverCompare ">=1.19-0" .Capabilities.KubeVersion.Version }}
apiVersion: networking.k8s.io/v1
{{- else }}
apiVersion: networking.k8s.io/v1beta1
{{- end }}
```

32. How do you reference a parent chart's values from a sub-chart? What about global values?

Direct parent values typically do not flow into sub-charts. Use global values:

```
global:
 storageClass: fast
,
```

In sub-chart:

```
{{ .Values.global.storageClass }}
```

33. What is the tpl function? Give a use case and explain why it must be used carefully.

tpl evaluates a string value as a Helm template. Use case: dynamic templates stored in values.

Example:

```
{{ tpl .Values.extraLabels . }}
```

Careful because it can execute arbitrary template expressions within values.

34. How do you iterate over a list in values.yaml using range? Give an example with environment variables.

Values:

env:

- name: ENVIRONMENT  
value: production
- name: LOG\_LEVEL  
value: debug

Template:

```
{{- range .Values.env }}
```

```
- name: {{ .name }}
```

```
value: {{ .value }}
```

```
{{- end }}
```

35. How do you iterate over a map/dictionary in values.yaml using range? Give an example.

Values:

labels:

- team: devops
- tier: backend

Template:

```
{{- range $key, $value := .Values.labels }}
```

```
{{ $key }}: {{ $value }}
```

```
{{- end }}
```

36. Write a template for a ConfigMap that reads from `.Values.config` as a map and renders each key-value pair.

```
apiVersion: v1
kind: ConfigMap
metadata:
 name: {{ include "mychart.fullname" . }}-config
data:
 {{- range $key, $value := .Values.config }}
 {{ $key }}: |-
 {{ $value | nindent 4 }}
 {{- end }}
```

37. How do you conditionally include a block in a template? Give an example with an optional Ingress.

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: {{ include "mychart.fullname" . }}
spec:
 rules:
 - host: {{ .Values.ingress.host }}
 http:
 paths:
 - path: /
 pathType: Prefix
 backend:
 service:
 name: {{ include "mychart.fullname" . }}
 port:
 number: {{ .Values.service.port }}
 {{- end }}
```

38. What is `values.schema.json`? How does it improve chart quality?

`values.schema.json` is a JSON Schema file that validates chart values. It improves chart quality by ensuring values conform to expected types, ranges, and required fields before install/upgrade.

39. Explain the lookup function. What are its limitations?

lookup queries live cluster objects during template rendering:

```
{{- $svc := lookup "v1" "Service" "default" "my-service" -}}
```

Limitations:

- only works during Helm install/upgrade with cluster access
- cannot be used with helm template in offline mode
- may increase deployment complexity and reduce reproducibility

40. How do you handle default values for nested objects in values.yaml?

Use default with nested paths:

```
replicas: {{ .Values.replicaCount | default 1 }}
```

image:

```
repository: {{ .Values.image.repository | default "nginx" }}
```

If nested objects may be nil, use default at each level or with blocks.

## COMMANDS & CLI QUESTIONS

41. What does helm list -A do? How is it different from helm list -n <namespace>?

helm list -A lists releases from all namespaces. helm list -n <namespace> lists releases only in the specified namespace.

42. Explain every column in the output of helm list.

- NAME: release name
- NAMESPACE: namespace of the release
- REVISION: current revision number
- UPDATED: last update timestamp
- STATUS: deployed, failed, deleted, superseded, pending-upgrade, etc.
- CHART: chart name and version
- APP VERSION: chart's appVersion
- NAMESPACE: release namespace

43. What does helm status <release> show? How is it different from helm get all?

helm status shows release status, namespace, revision, notes, and summary info. helm get all returns all release metadata, manifests, values, hooks, and notes in full.

44. What is the difference between `helm get values` and `helm get values --all`?

`helm get values` shows user-supplied values. `helm get values --all` shows both user-supplied and computed default values.

45. How do you install a chart with a custom values file AND override one additional value on the command line?

```
helm install myapp ./chart -f prod-values.yaml --set replicaCount=4
```

46. How do you upgrade a release and ensure it rolls back automatically on failure?

```
helm upgrade myapp ./chart -f prod-values.yaml --atomic --wait --timeout 5m
```

47. How do you render a chart's templates locally without a cluster?

```
helm template myapp ./chart -f values.yaml
```

48. How do you install a specific version of a chart?

```
helm install myapp bitnami/nginx --version 9.3.12
```

49. How do you pass a multi-line string as a `--set` value?

Use quoted YAML or `--set-file` when possible. Example with escaped newlines:

```
helm install myapp ./chart --set-string config="line1\nline2\nline3"
```

50. How do you see what manifest a currently deployed release actually contains?

```
helm get manifest myapp
```

51. What does `helm history <release>` output? Explain the STATUS column values.

It outputs revision history records. STATUS values include:

- **deployed**: currently deployed revision
- **superseded**: replaced by a newer revision
- **failed**: installation or upgrade failed
- **pending-upgrade**: upgrade in progress
- **pending-install**: install in progress
- **uninstalled**: release was uninstalled

52. How do you rollback to revision 2 of a release?

```
helm rollback myapp 2
```

53. How do you uninstall a release but keep its history?

```
helm uninstall myapp --keep-history
```

54. How do you install a chart with a generated release name?

```
helm install ./chart --generate-name
```

55. How do you search for all versions of a specific chart in a repository?

```
helm search repo nginx --versions
```

## HOOKS QUESTIONS

56. What are Helm hooks? List all hook types.

Helm hooks are special resources that run at lifecycle events. Hook types:

- pre-install
- post-install
- pre-upgrade
- post-upgrade
- pre-delete
- post-delete
- pre-rollback
- post-rollback
- test

57. How do you define a Kubernetes Job as a pre-upgrade hook?

```
apiVersion: batch/v1
```

```
kind: Job
```

```
metadata:
```

```
 name: "{{ include \"mychart.fullname\" . }}-migrate"
```

```
 annotations:
```

```
 "helm.sh/hook": pre-upgrade
```

```
spec:
```

```
 template:
```

```
 spec:
```

```
 containers:
```

```
 - name: migrate
```

```
 image: alpine
```

```
 command: ["sh", "-c", "echo migrate"]
```

```
 restartPolicy: Never
```

58. What is helm.sh/hook-weight? How does it control hook execution order?

helm.sh/hook-weight is an integer used to order hooks with the same type. Lower weights run



first.

59. What is helm.sh/hook-delete-policy? Explain each possible value.

- before-hook-creation: delete old hook resources before creating new ones
- hook-succeeded: delete the hook resource when it succeeds
- hook-failed: delete it when it fails

Use it to avoid accumulating hook resources.

60. What happens to a release if a hook fails?

If a required hook fails, the release may become failed or stay in a pending state. A failed hook can block installation or upgrade.

61. How do you write a Helm test? What annotation does a test pod need?

Add annotation:

```
metadata:
 annotations:
 "helm.sh/hook": test
```

Then run:

```
helm test myapp
```

62. What is the difference between a hook and a regular template resource?

A hook is executed during lifecycle events and may create jobs or other resources specifically for the hook. Regular template resources are managed as part of the release and are reconciled normally.

## SECURITY QUESTIONS

63. How do you read a Helm release secret with kubectl? Write the full command sequence to decode it.

```
kubectl get secret -n production
kubectl get secret sh.helm.release.v1.my-release.v1 -n production -o jsonpath="{.data.release}" |
base64 --decode | base64 --decode | gunzip -c
```

64. Why is `--set password=mypassword` insecure? What are the secure alternatives?

It exposes the password in shell history and process lists. Alternatives:

- `--set-file password=/path/to/file`
- encrypted values with Helm Secrets and SOPS
- external secret management systems

65. What is the Helm Secrets plugin? How does it work with Mozilla SOPS?

The Helm Secrets plugin encrypts values files with SOPS and decrypts them at deploy time. It keeps secrets encrypted in Git and allows Helm to read decrypted values locally.

66. What RBAC permissions does a service account need to perform helm install?

At minimum, permissions to create/update/delete the Kubernetes resources in the target namespace and create the release storage object (Secret or ConfigMap). If using cluster-level resources, cluster role permissions are required.

67. How do you sign a Helm chart? What is a provenance file?

Sign a chart:

```
helm package ./chart --sign --key mykey --keyring ~/.gnupg/secring.gpg
```

A provenance file is a `.prov` file that verifies the chart signature and integrity.

68. How do you verify a signed chart with helm verify?

```
helm verify mychart-0.1.0.tgz
```

## DEPENDENCIES QUESTIONS

69. How do you add a dependency to a chart? Show the Chart.yaml syntax.

dependencies:

- name: redis
- version: 19.10.1
- repository: <https://charts.bitnami.com/bitnami>

70. What is the difference between helm dependency update and helm dependency build?

helm dependency update updates the Chart.lock file and downloads dependencies. helm dependency build rebuilds the charts/ directory from Chart.lock without modifying it.

71. How do you enable/disable a sub-chart dependency using a condition?

In values.yaml:

```
redis:
 enabled: false
```

In Chart.yaml:

```
- name: redis
 condition: redis.enabled
```

72. How do you pass values to a sub-chart from the parent?

Use the sub-chart key in the parent values.yaml:

```
redis:
 auth:
 password: secret
```

73. What are global values and how do they work across chart dependencies?

global values are shared across a parent chart and all sub-charts.

Example:

```
global:
 storageClass: fast
```

All child charts can reference .Values.global.storageClass.

## REPOSITORY QUESTIONS

74. How do you add the Bitnami Helm repository?

```
helm repo add bitnami https://charts.bitnami.com/bitnami
```

75. How do you search for an nginx chart across all public repositories?

```
helm search hub nginx
```

76. How do you pull a chart to inspect it locally without installing?

`helm pull bitnami/nginx --version 9.3.12`

77. How do you create and host your own Helm chart repository on GitHub Pages?

78. Package the chart: `helm package ./chart`

79. Generate index: `helm repo index . --url https://<user>.github.io/<repo>`

80. Push .tgz and index.yaml to gh-pages or docs/

81. Enable GitHub Pages for the repository

82. What is `helm repo update` and when should you run it?

`helm repo update` refreshes local repo cache from remote repositories. Run it before searching or installing charts from a repository.

## CI/CD & PRODUCTION QUESTIONS

79. Describe a production-safe Helm upgrade pipeline. What commands run in what order?

80. `helm lint ./chart`

81. `helm template myapp ./chart -f values-prod.yaml`

82. `helm template myapp ./chart -f values-prod.yaml | kubeconform`

83. `helm diff upgrade myapp ./chart -f values-prod.yaml`

84. `helm upgrade --install myapp ./chart -f values-prod.yaml --atomic --wait --timeout 5m`

85. What is Helmfile? How is it different from running multiple `helm install` commands?

Helmfile is a declarative tool that defines multiple releases in a single YAML. It manages release dependencies and environments in one place, unlike separate `helm install` commands.

81. How does ArgoCD use Helm? Does ArgoCD run `helm install` directly?

ArgoCD uses Helm as one of its application sources. It may not invoke the CLI directly; instead it uses Helm libraries to render charts and deploy resources.

82. What is the `helm diff` plugin and why is it essential before production upgrades?

`helm diff` shows the actual changes between the current release and the proposed upgrade. It is essential to catch unexpected resource updates before applying them.

83. How do you promote a release from dev to prod with different values?

Use separate values files and release names:

`helm upgrade --install myapp-prod ./chart -f values-prod.yaml`

84. What is the recommended way to handle image tags across environments?

Use explicit tags per environment, not latest. Store them in values files per environment and use CI to update tags deterministically.

85. How do you enforce that all production charts have resource limits defined?

Use `values.schema.json` to require `resources.requests` and `resources.limits`, and linting checks in CI.

## SCENARIO / TROUBLESHOOTING QUESTIONS

86. You run `helm install myapp ./myapp-chart` and it hangs. How do you diagnose and fix this?

Check `helm status myapp`, cluster events, and pod status. Use `kubectl describe pod -n <ns>` and `kubectl logs` to find readiness or image pull issues.

87. You run `helm upgrade myapp ./myapp-chart` and get `Error: UPGRADE FAILED: another operation (install/upgrade/rollback) is in progress`. What caused this and how do you fix it?

A prior Helm operation is still active or timed out. Wait for completion, inspect `helm history myapp`, and if necessary use `helm rollback` or remove the stuck release state.

88. Your `helm rollback` fails with `Error: release has no deployed releases`. What does this mean and how do you recover?

It means there is no prior successful deployed revision. Recover by installing a fresh release or using a known good chart version if available.

89. A hook job is failing and blocking your release. How do you clean it up and proceed?

Inspect the hook job with `kubectl describe job` and logs. If safe, delete the failed hook job and rerun the Helm command or `rollback`.

90. You need to see exactly what YAML Helm will apply before running `helm upgrade`. What command do you use?

```
helm template myapp ./myapp-chart -f values.yaml
```

91. A colleague deleted a Kubernetes deployment that was part of a Helm release. `helm status` shows deployed. How do you reconcile the state?

Use `helm get manifest myapp` to inspect the release manifest and then reapply or upgrade the release. If resources are missing, a `helm upgrade --install` can recreate them.

92. You accidentally ran `helm uninstall` on a production release. What options do you have to recover?

If history was kept, reinstall from the same chart and values or use `helm rollback` on the previous release name if the release still exists in history. If resources are truly deleted, restore from backups or recreate the release.

93. A chart template fails with Error: template: mychart/templates/deployment.yaml:15:4: executing "mychart/templates/deployment.yaml" at <.Values.image.tag>: nil pointer evaluating interface {}.tag. How do you fix this?

Ensure `.Values.image` is defined in `values.yaml` and use `default` or `required` to protect against missing values.

94. You want to install the same chart twice in the same namespace with different configurations. How?

Use different release names:

```
helm install myapp-dev ./chart -f dev-values.yaml
```

```
helm install myapp-prod ./chart -f prod-values.yaml
```

95. Your chart works in dev but fails in prod because the prod cluster doesn't have a certain CRD. How do you make the chart handle this gracefully?

Use `lookup` or conditional templates to detect CRD availability, or split CRD installation into a separate pre-install step. Provide a clear message if the CRD is missing.

## Chapter 19 summary

- Covered conceptual, architecture, templating, CLI, hooks, security, dependency, repository, CI/CD, and troubleshooting questions.
- Practice writing Helm commands and understanding Helm internals for exam readiness.

- Remember the difference between Helm charts, releases, revisions, values precedence, and hooks.

## PART 7: QUICK REFERENCE

### Chapter 20 — Helm Cheat Sheet

#### Common Helm commands

```
helm version
helm env
helm repo add <name> <url>
helm repo update
helm repo list
helm search repo <keyword>
helm search hub <keyword>
helm install <release> <chart> [flags]
helm upgrade <release> <chart> [flags]
helm rollback <release> <revision>
helm uninstall <release> [--keep-history]
helm list [-A] [-n <namespace>]
helm status <release>
helm get all <release>
helm get values <release> [--all]
helm get manifest <release>
helm get hooks <release>
helm get notes <release>
helm template <release> <chart> [flags]
helm lint <chart>
helm test <release>
helm package <chart>
helm verify <chart.tgz>
helm repo index <directory> --url <url>
helm dependency update <chart>
helm dependency build <chart>
helm plugin list
```

#### Most-used Helm flags

- --namespace / -n
- --create-namespace
- --values / -f

- --set
- --set-string
- --set-file
- --dry-run
- --debug
- --wait
- --timeout
- --atomic
- --reuse-values
- --reset-values
- --install
- --generate-name
- --version
- --devel
- --dependency-update

## Helm environment variables

- HELM\_CACHE\_HOME: cached charts and repository data
- HELM\_CONFIG\_HOME: Helm config files and repositories
- HELM\_DATA\_HOME: Helm local data and plugin storage
- HELM\_DRIVER: storage driver for release state
- HELM\_KUBECONTEXT: kubeconfig context to use
- HELM\_NAMESPACE: default namespace for Helm commands
- HELM\_DEBUG: enable debug logging
- HELM\_NO\_PLUGINS: disable plugin loading
- HELM\_PLUGINS: override plugin directory
- HELM\_REPOSITORY\_CACHE: repository cache location
- HELM\_REPOSITORY\_CONFIG: repository config location

## Chart directory structure at a glance

Chart.yaml  
values.yaml  
charts/  
templates/  
  deployment.yaml  
  service.yaml  
  ingress.yaml  
  \_helpers.tpl  
NOTES.txt  
tests/test-connection.yaml



.helmignore

## Built-in template objects quick reference

- .Release: release metadata
- .Chart: chart metadata
- .Values: chart values
- .Files: chart files
- .Capabilities: Kubernetes API and version info
- .Template: template metadata

## Most-used template functions

- default: fallback value
- required: fail if missing
- toYaml: convert object to YAML
- fromYaml: parse string to YAML
- indent: indent block
- nindent: newline + indent
- quote: quote values
- include: render named template
- tpl: render dynamic template string
- lookup: query live cluster objects

## Values precedence order

1. values.yaml
2. sub-chart values
3. parent chart values
4. -f values files
5. --set

## Hook types

- pre-install
- post-install
- pre-upgrade
- post-upgrade
- pre-delete
- post-delete
- pre-rollback
- post-rollback

- test

## Hook delete policies

- before-hook-creation
- hook-succeeded
- hook-failed

## Common error messages and fixes

- another operation is in progress: wait or resolve stuck release state
- cannot re-use a name that is still in use: uninstall or use --replace
- release has no deployed releases: no prior deployed revision exists
- nil pointer evaluating interface: missing values or nil paths in templates
- HOOKS FAILED: inspect hook resources and logs
- DOWNLOAD FAILED: check repository URL and network

## Chapter 20 summary

- Use this cheat sheet to quickly recall Helm commands, flags, environment variables, chart structure, template objects, values precedence, hook types, and common fixes.
- Keep the sheet handy during practice and review for certification exam preparation.